Assignment 1 Reports - FIT2099

Written by: Nguyen Khang Huynh - 33326460

Table of Contents

Requirement 1

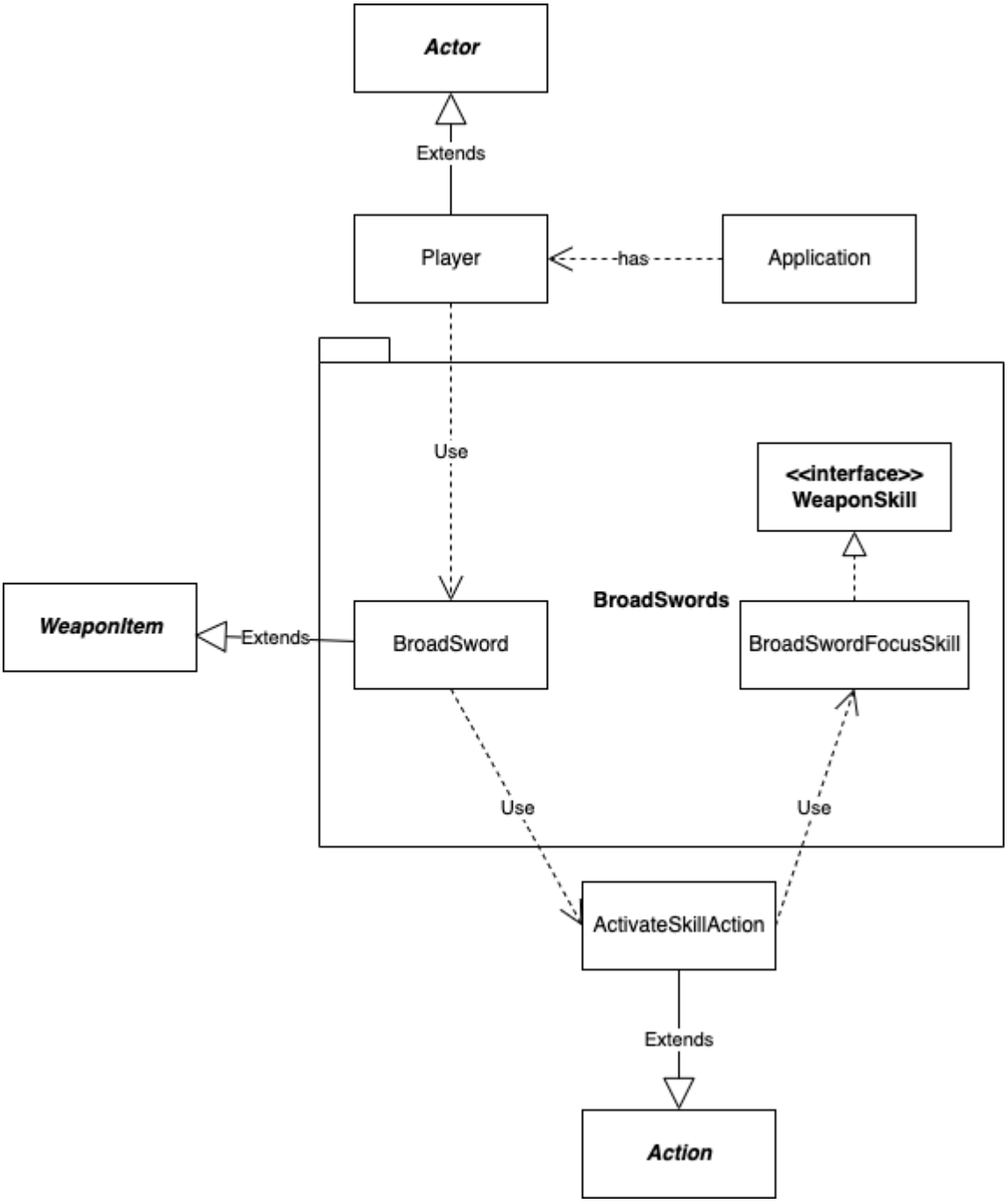

Photo 1: Requirement 1 UML

In summary, requirement 1 asks to create a player, an actor with health and stamina spawn in the building of the map. In addition, the player's stamina must recover 1% of their maximum after one turn. Furthermore, in the building, there is also a weapon called "BroadSword," which has a skill named "Focus," this skill will reduce the player's stamina by 20% of their maximum stamina and be deactivated after five turns. In order to achieve this, I have come up with the ideas fully described in the UML above.

In order to update the player's stamina, I increased the player's stamina in the method "play turn," which overrides the Actor class since this method will be executed in each turn. Doing this will make sure my player's stamina constantly gets updated. This way can easily be extended if the player gains more ability, which needs to be changed each turn.

In order to reduce 20% of their maximum stamina to the current stamina, I will reduce the stamina in the ActivateSkillAction whenever this action is called.

In order to deactivate when it is five turns or out of inventory, I use the tick method to take responsibility for counting inside the inventory and realise when the item is out of the inventory.

In order to meet the requirements of SOLID and DRY principles, I decided to create two classes: the activateSkillAction and the BroadSwordFocusSkills for the BroadSword. As seen in the UML, the activateSkillActon is extended from the Action class, and it helps activate the Focus skills whenever the user calls. The BroadSwordFocusSkills are extended from the WeaponSkills interface and implemented to change the user status. The ideas help implement the Single Responsibility Principle since two classes are responsible for each idea. Not only that, it also meets the Open/Closed Principle because if you want to extend the "Focus" skill to increase more status, you can add a new method to the implementation without modifying any previous implementation. By using the interface for the BroadSwordFocusSkills, you will meet the Liskov Substitution Principle since you can use the WeaponSkills interface class to replace the BroadSwordFocusSkills, which is helpful if, in the future, you want to extend the BroadSword by adding more skills. Using the interface and abstract classes will also meet the DRY principles since the code will not repeat each other whenever you extend.

The disadvantage of this design will be the system's complexity since more classes are created and more relationships to manage. The new developer needs to take more time to understand the code briefly.

Requirement 2

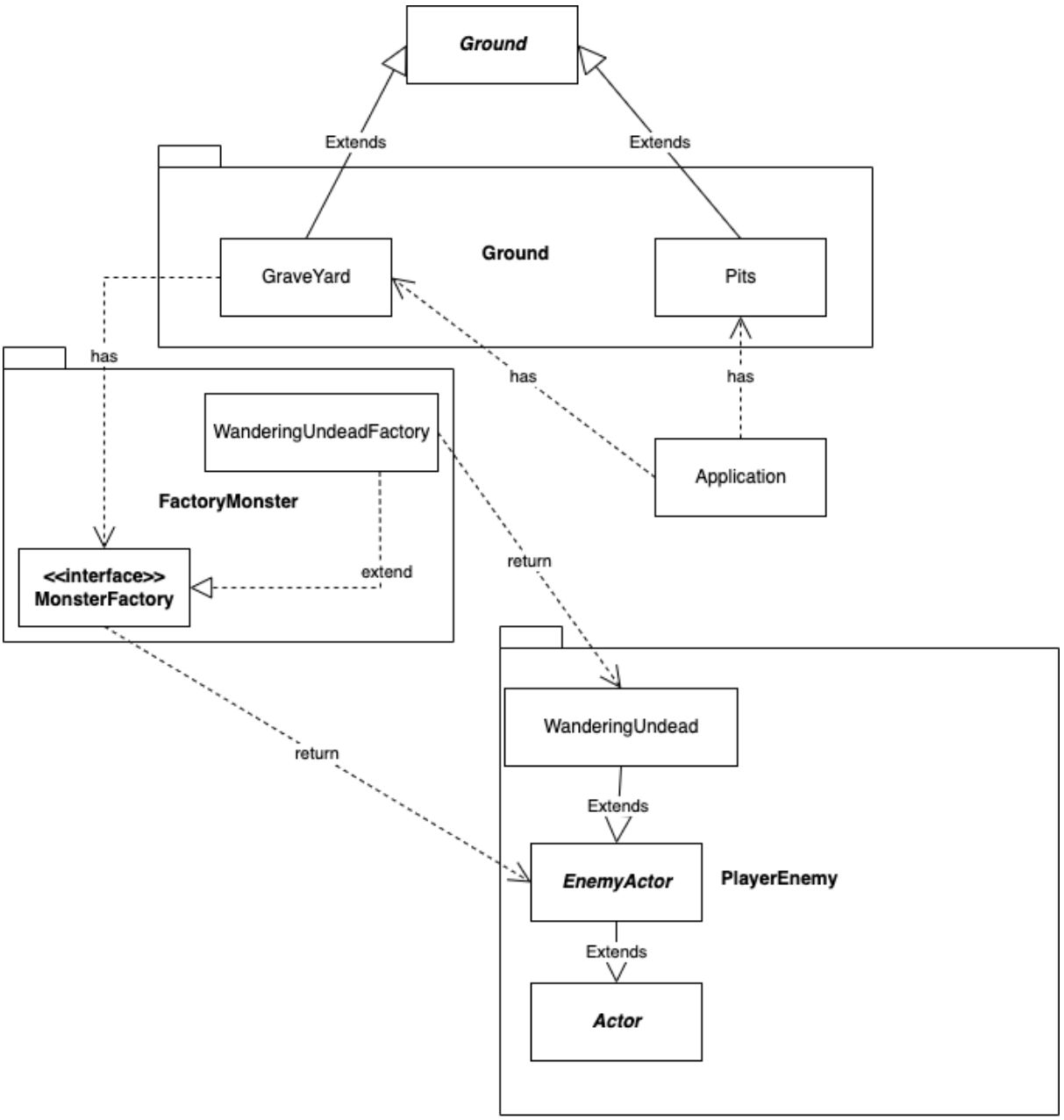

Photo 2: Requirement 2 UML

In summary, requirement 2 asks to create a Pit (Voids) and a Graveyard extended from the Ground class in the game Engine. The Pit is responsible for killing all actors who step on it, and if the player steps on it, it has to display the fancy message: "You died." Furthermore, the Graveyard has to spawn the Enemy with a 25% chance. In order to achieve this, I have come up with the ideas fully described in the UML above.

In order to eliminate the Actor, the pits will override the tick method of the ground class to check if any actor stands on it in each turn. It will execute the "unconscious" methods of the Actor who steps on it, and the Actor will be removed from the map. And we can implement the "unconscious" of the player to display the Fancy Message whenever the player dies.

In order to meet the requirements of the SOLID and DRY principles, I created the interface "MonsterFactory" and the class "WanderingUndeadFactory." The MonsterFactory has two methods: to give out the Enemy Actor class and the chance of spawning the Enemy Actor. The "WanderingUndeadFactory" class is designed to focus only on generating the Wandering Undead in the game. Doing this will help the class responsible for doing a single job and can add new methods without modifying the current implementation. Overall, the design will help extend the Graveyard's implementation, as witnessed in Requirement 5.

The disadvantage of this design will be the system's complexity since more classes are created and more relationships to manage. This implementation is extendable; however, in the situation where the developer wants to make a want to spawn an Actor that is not an Enemy Actor, this idea cannot be used and has to create a new class for it.

Requirement 3

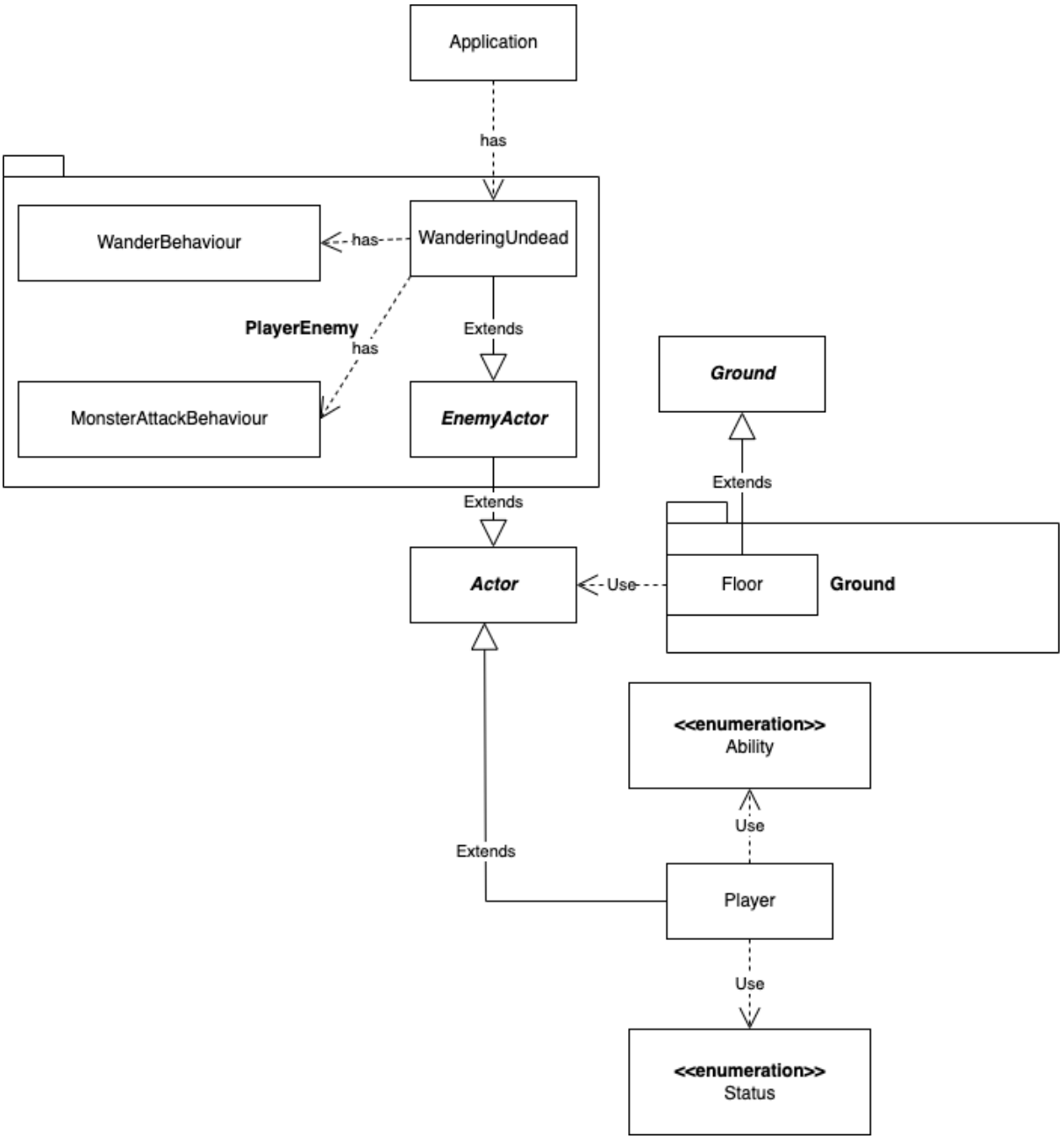

Photo 3: Requirement 3 UML

In summary, requirement 3 asks to modify the Wandering Undead so that it attacks the Player in the distance of one block. It cannot attack other enemies and cannot enter the floor. In order to achieve this, I have come up with the ideas fully described in the UML above.

In order to avoid the enemy attacking each other, I created an enumeration Status and named the value "HOSTILE_TO_ENEMY," and added them to the Player. The enum will help the enemy attack only the Actor who contains it. This status can be reused if, in the future, we create new actors that are enemies of those Enemy actors and extendable if we need to add more types of Actors.

The same ideas as above, by modifying the floor to allow only actors containing the value "ALLOWABLE_TO_ENTER" of the Ability enumeration, we can easily avoid any enemy actor trying to get in the floor. It can be used for multiple purposes and any actor in the future extension.

A noticeable design detail is the appearance of the new Abstract class Enemy Actor, which extends from Actor. Instead of the Wandering Undead Class extended from Actor, I let the Actor extend the Enemy Actor class since it can help for future extension if we create more opponent Actors that counter the Player. The MonsterAttackBehaviour, which extends from the Behaviour class in the game Engine, takes responsibility for checking and gives the decision of attack to the Enemy Actor.

The disadvantages of this design are the appearance of many classes and relationships to manage. Also, the Enumeration class might need to be clarified for those new developers when they join the project, leading them to end up using the wrong purpose of Enumeration.

Requirement 4

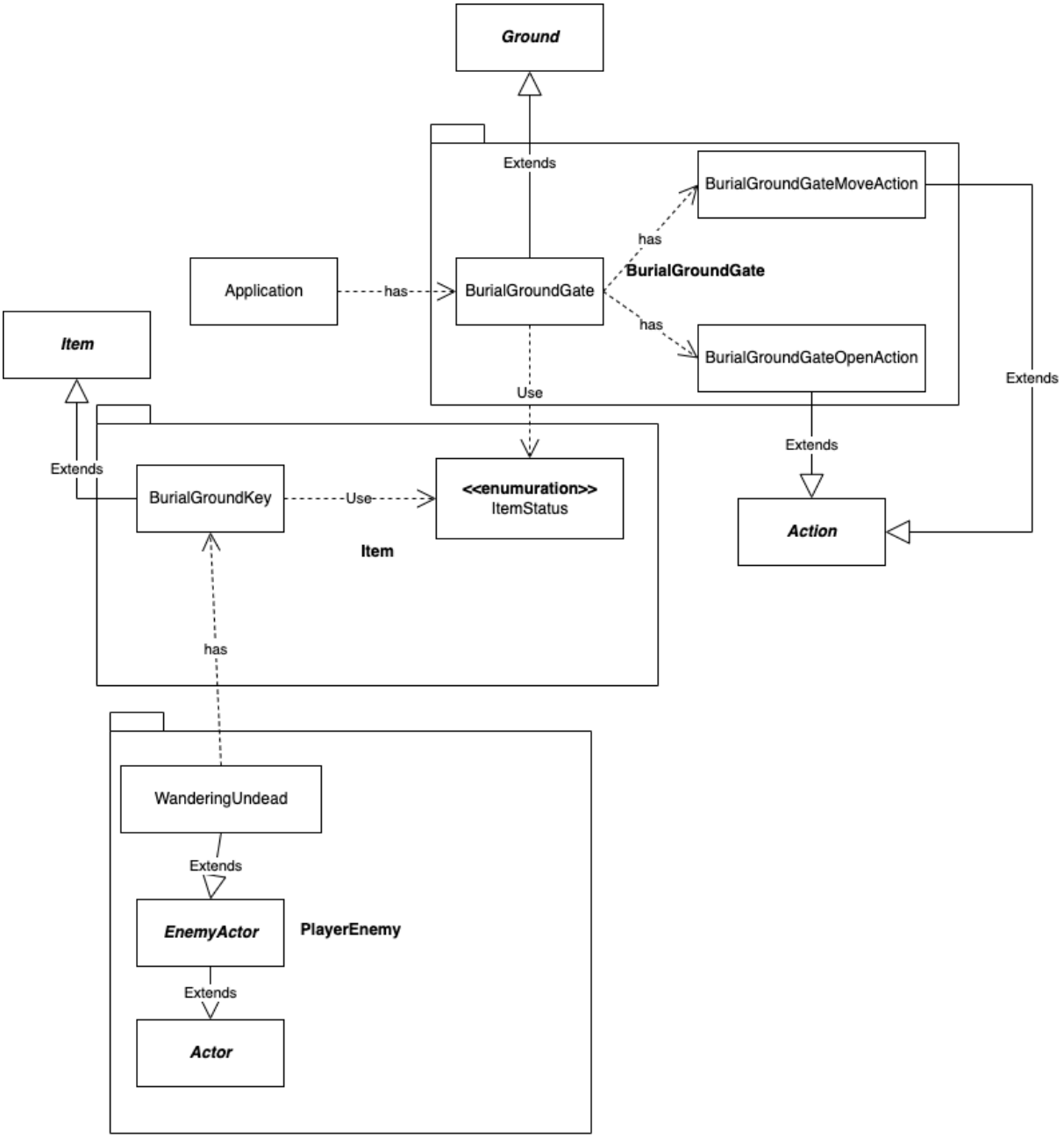

Photo 4: Requirement 4 UML

In summary, requirement 4 asks to create a Key class to let the Player be able to open the Gate to move to the Burial Ground map. In order to achieve this, I have come up with the ideas fully described in the UML above.

First, I created the BurialGroundKey class, extending the Item class and adding the value of the Enumeration class named Item Status. This implementation will help the BuiralGroundGate to sort out the BurialGroundKey and let it activate two actions of the Gate, which are BurialGateOpenAction and BurialGateMoveAction.

Second, the BurialGroundGate is implemented to extend and can be used on any map, and they are unique so that the Player can be moved to the correct game map. Next, I created two different classes, one class for open and one class for moving the actor to the following map, and both of them extended to the Action class. The BurialGateOpenAction class is responsible for changing the door's status, so the system will ask the Player to move to a new map. The BurialGateMoveAction will remove the actor from the current map and immediately add them to the corresponding map so that the system will not think the Player is dead and stop the game.

The advantages of this design will help the code to be extended for future creation of more maps since the Gate can be defined as unique and the same as the key. Each function of the Gate is responsible for a single action, which can be added without changing the current code.

The disadvantages of this design are also the appearance of many classes and relationships to manage. Furthermore, adding items one item at a time is ineffective when the Enemy Actor contains hundreds of items.

Requirement 5

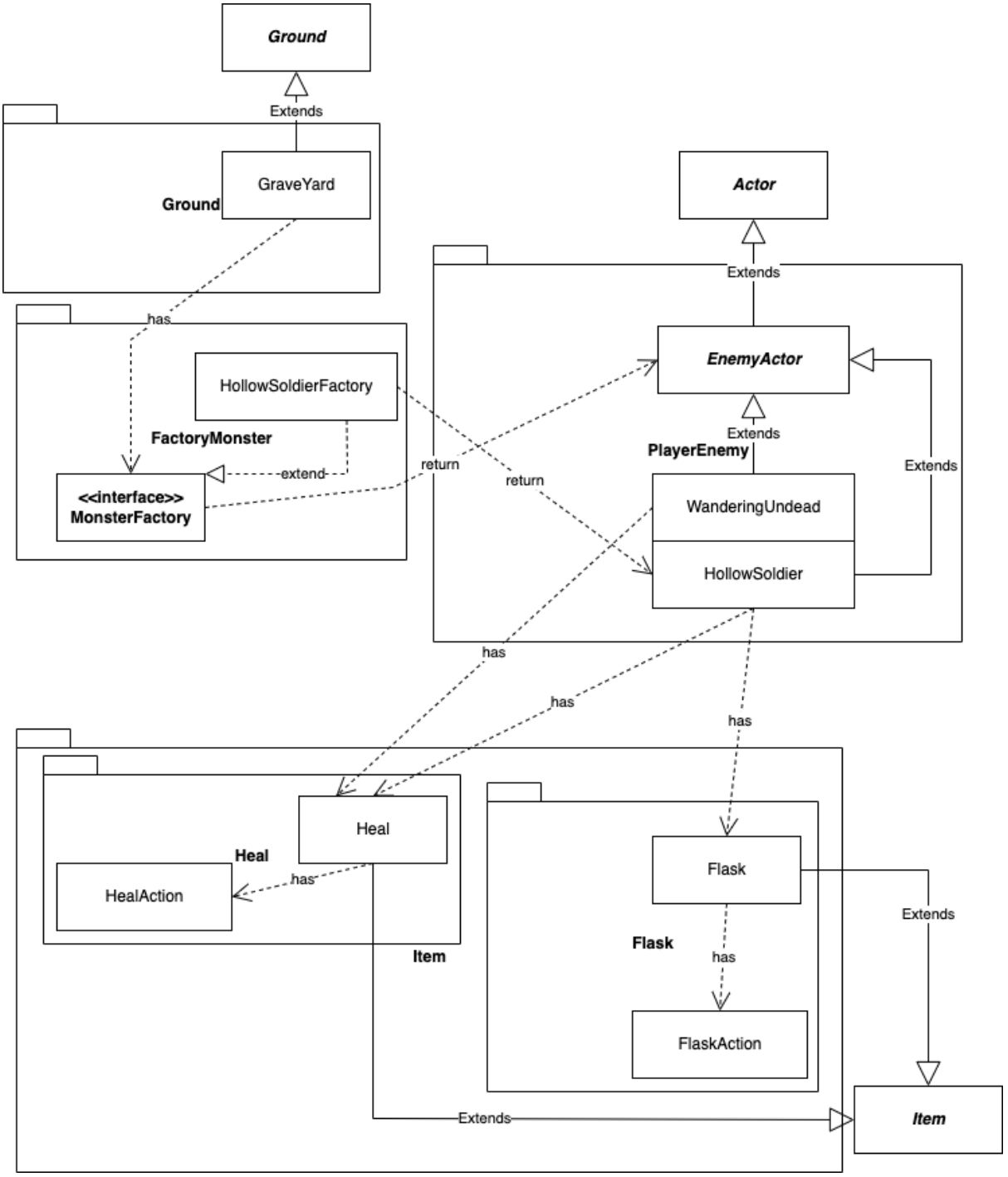

Photo 5: Requirement 5 UML

In summary, requirement 5 asks to create a new enemy actor named Hollow Soldier, and it has to contain two more items, the Healing Vial and the Refreshing Flask, and add the Healing Vial to the Wandering Undead class. In order to achieve this, I have come up with the ideas fully described in the UML above.

From Requirement 3, I extended the Enemy Actor to the Hollow Soldier since the implementation of the Hollow Soldier is repeated to the Wandering Undead, which will help the design avoid repetition. The Healing and Flask classes are extended from the Item classes in the game engine. Moreover, I create each class an action so that the class does not have to handle multiple tasks.

The advantage of this design is to reduce the repetition of the code. Not only that, the design also gives each class a single responsibility, and each task has a strong relationship with the other.

The disadvantages of this design are also the appearance of many classes and relationships to manage. Furthermore, adding items one item at a time is ineffective when the Enemy Actor contains hundreds of items.